Together First, Apart Later? When Optimizers Start to Learn Different Functions

Initial checkpoint, MIT 6.7910, Fall 2026

Juntang Wang and Víctor Conchello · October 30, 2026

Question. From the same initialization and the same sequence of mini-batches, do SGD with momentum, AdamW and Muon first follow the same path in function space and only later separate? If so, where is the split, what coincides with it (an edge-of-stability event (Cohen et al., 2021; Cohen et al., 2022), the end of warmup, a loss level), and is the final function already decided there?

What we have done. We trained nanoGPT (Karpathy, 2022) on character-level Shakespeare (10.8M parameters) and TinyStories-10M and -30M (14M and 30M parameters) with SGD, AdamW and Muon (Jordan et al., 2024), at learning rates from a sweep, from a shared initialization and with five data orders per optimizer. Along training we logged the next-token probability distributions on fixed held-out contexts, and the top eigenvalue of the Hessian and, for AdamW, of the preconditioned Hessian (Cohen et al., 2022), both computed with the Lanczos method (1950). We swapped optimizers at nine fork steps with warm optimizer states, measured linear mode connectivity, and replicated the CIFAR-10 break-even analysis (Jastrzebski et al., 2020). In total 404 runs, about 105 GPU-hours.

Method. The distance between two runs is the mean Jensen–Shannon divergence of their next-token predictions. We compare runs at equal *training loss*, so that one optimizer being faster is not counted as a difference, and divide by a noise floor: the distance between two runs of the same optimizer that differ only in data order. A split is a stretch of loss levels where this ratio exceeds 1.5 for three consecutive levels (a hand-set margin; 1.25, 2 and another metric give the same conclusions); it is persistent if it lasts to the end of training, with a bootstrap over data orders for its location.

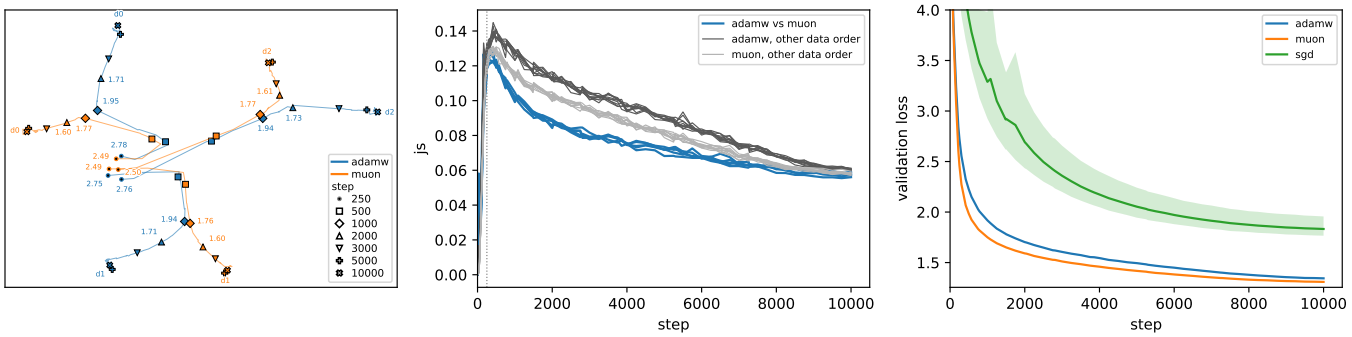


Figure 1: TinyStories-10M. Left: the trajectories of the AdamW and Muon runs on three data orders (d0–d2, at the end of each run) from step 250 on: densMAP of the Jensen–Shannon distance (the square root of the divergence) between the predictions of every pair of logged steps. Black circles: step 250; markers: fixed steps, with each run's training loss written at steps 250, 1000 and 2000. densMAP keeps neighbourhoods but not distances. Middle: divergence between AdamW and Muon at equal steps (blue) and the data-order floor (grey); dotted: step 250, where the left panel starts. Right: validation loss of AdamW, Muon and SGD, mean over five data orders, band from min to max. Hyperparameters in Appendix A.1.

Result 1: at equal loss, no split after warmup (Fig. 2). After warmup, the distance between optimizers at equal loss stays at about the data-order floor to the end of training, in all three settings and for every pair except Muon–SGD on Shakespeare, where SGD barely trains; that split is lost with another metric or a stricter threshold. The only stretches above the floor are brief and within warmup, where the floor is near zero because the runs share their initialization. Muon is faster (Fig. 1, right; AdamW needs 1.35 and 1.22 times as many steps as Muon to reach its final validation loss on TinyStories-10M and -30M), yet at equal loss it lands on the same function as AdamW up to data-order noise: at the end of TinyStories-30M, replacing AdamW by Muon changes the predictions no more than changing the data order. After warmup (Fig. 1, left), AdamW and Muon with the same data order are nearest neighbours in the embedding, with Muon at lower loss at each step, yet at most about a fifth closer in Jensen–Shannon divergence than two data orders of one optimizer (0–23%, steps 250–3000), from their shared recent batches; at equal loss this vanishes. So the answer to the title is no: after warmup we find no split, early or late; the optimizers' functions are as close as the data order lets them be.

Result 2: the last optimizer decides the function (Fig. 3). When we switch optimizer at step t, the fork ends within the floor of the new optimizer whenever that optimizer is fast enough to get there in the remaining steps: forks from SGD to AdamW or Muon land on the new optimizer even after 4000 of 10,000 steps, while forks into the slower SGD end closer to their parent from step 1500 on (step 5000 on TinyStories-30M). The endpoint is thus not decided

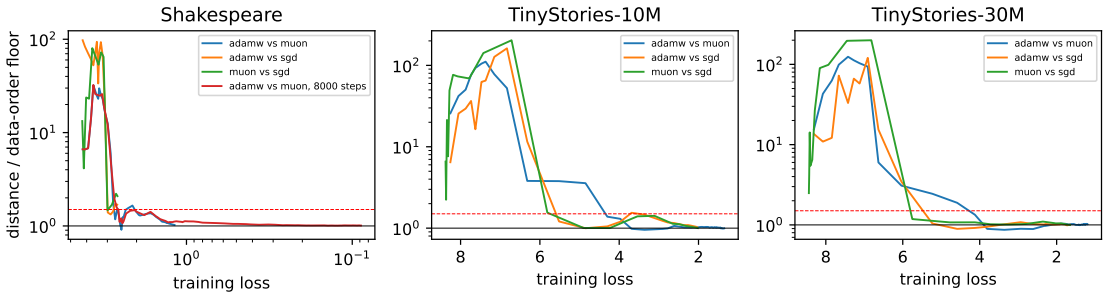


Figure 2: Loss-matched distance between optimizers over the data-order floor (training progresses to the right); red dashed, the threshold 1.5. Red, left: AdamW–Muon trained into memorization (8000 steps). Hyperparameters in Appendix A.2.

early. Weight space keeps a mark that function space does not: after a change of optimizer, a fork needs a later fork step to stay linearly connected to its parent (Frankle et al., 2020) than after a change of data order.

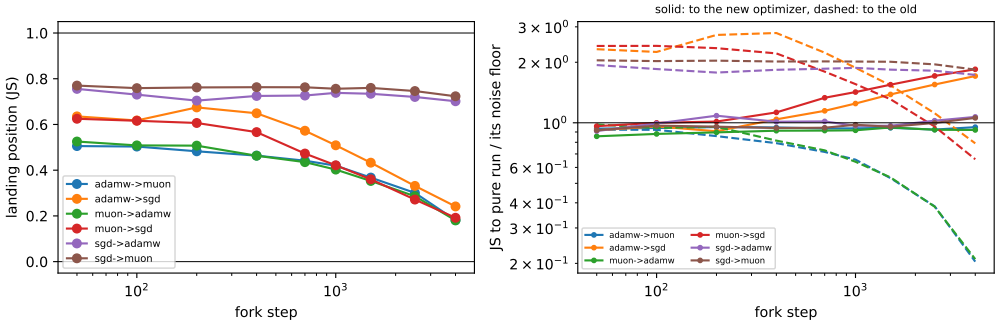


Figure 3: Swaps on TinyStories-10M. Left: where the fork ends between pure A (0) and pure B (1). Right: its distance to pure B (solid) and to pure A (dashed) over their floors; below 1 means indistinguishable from it. Hyperparameters in Appendix A.3.

Result 3 (moonshot): the sharp direction at the edge of stability. Restricted to the steps at each optimizer's threshold and averaged over the five data orders, the top eigenvector of the operator that sets the threshold lies at the input end: the embeddings and first three blocks carry 75–90% of it in all settings. SGD's lies mostly in the first block; AdamW's, of $P^{-1/2}HP^{-1/2}$, in the embeddings on TinyStories (about 60%), where rare tokens make P^{-1} largest; Muon's, without a threshold, in its first three blocks (on Shakespeare in the embeddings, left to its auxiliary AdamW). Except in the embeddings, 66–83% of the eigenvalue comes from curvature *between* blocks. The direction changes block long after warmup, when the optimizers are already at the floor, so it does not mark a split.

Result 4: a model of the results (proof sketch). Let $\mathcal{F}_s$ be the predictors that see only the last s tokens, with best element q_s. As $\sqrt{\mathrm{JS}}$ is a metric, JS (bits) $\leq$ total variation, and by Pinsker, two predictors in $\mathcal{F}_s$ with excess losses $\Delta L_A, \Delta L_B$ over q_s have $\mathbb{E}\,\mathrm{JS} \leq \sqrt{2}(\sqrt{\Delta L_A} + \sqrt{\Delta L_B})$, whatever the optimizer. So at equal loss the bound forces the optimizers together near the bigram optimum and below, but not at higher loss, where their routes can differ, as in the warmup stretches (SGD passes through the unigram model, Muon skips it), nor in a class without a best element, as in the linear control below. The break-even point depends on how the update scales with the gradient: $(2+2\mu)/\eta$ for SGD; for AdamW the threshold applies to $P^{-1}H$, so the raw sharpness there scales as the gradient over η, as we find (log-log slope 1.15–1.6 on TinyStories); Muon's update is invariant to the gradient's scale and on a quadratic never diverges, so it has no break-even point on its matrices. Stability caps curvature but does not choose between functions of equal loss.

Pipeline check and controls. On CIFAR-10 the break-even effects replicate: the sharpness levels off at or below $2/\eta$ and lower for larger rates, and the gradient covariance is better conditioned; but at equal loss two learning rates differ by at most about twice the floor. Controls: a second initialization reproduces the results, and on a linear model on separable data, where gradient descent and Adam have different max-margin limits (Soudry et al., 2018; Zhang et al., 2024), the same test does find a persistent split.

Next (Nov–Dec). (i) Moonshot: whether the sharp direction moves to the new optimizer's layers after a swap, and lowering the learning rate of only those layers. (ii) Sweeps with sharpness (running): Muon's should not scale as $1/\eta$. (iii) Presentation (Dec 1–4), paper, code and Colab notebook (Dec 5–11). Code: `https://github.com/qqgjyx/MIT-6.7910`.

References

Cohen, J. M., B. Ghorbani, S. Krishnan, et al. (2022). *Adaptive Gradient Methods at the Edge of Stability*. Preprint.

Cohen, J. M., S. Kaur, Y. Li, et al. (2021). "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability". In: *ICLR*.

Frankle, J., G. K. Dziugaite, D. M. Roy, and M. Carbin (2020). "Linear Mode Connectivity and the Lottery Ticket Hypothesis". In: *ICML*.

Jastrzebski, S., M. Szymczak, S. Fort, et al. (2020). "The Break-Even Point on Optimization Trajectories of Deep Neural Networks". In: *ICLR*.

Jordan, K., Y. Jin, V. Boza, et al. (2024). *Muon: An Optimizer for Hidden Layers in Neural Networks*. Blog post. URL: https://kellerjordan.github.io/posts/muon/.

Karpathy, A. (2022). *nanoGPT*. Software. URL: https://github.com/karpathy/nanoGPT.

Lanczos, C. (1950). "An Iteration Method for the Solution of the Eigenvalue Problem of Linear Differential and Integral Operators". In: *Journal of Research of the National Bureau of Standards* 45.4, pp. 255–282.

Soudry, D., E. Hoffer, M. S. Nacson, et al. (2018). "The Implicit Bias of Gradient Descent on Separable Data". In: *JMLR*.

Zhang, C., D. Zou, and Y. Cao (2024). "The Implicit Bias of Adam on Separable Data". In: *NeurIPS*.

A Hyperparameters

A.1 Runs of Figure 1

Table 1: Shared by the 15 TinyStories-10M runs. Values from the repository's configuration files and code.

Runs	3 optimizers × data orders d0–d4 (15)
Model	nanoGPT, pre-LayerNorm blocks
Layers, heads, width	6, 6, 384
MLP	GELU, width 1536
Biases, dropout	none, none
Output head	untied
Attention projections	separate query, key, value
Context	256 tokens
Parameters	13.9M (10.6M without token and position embeddings and head)
Data	TinyStories (GPT-4 version)
Tokenizer	byte-level BPE, 4096 tokens, trained on the training split
Batch	128 windows × 256 tokens
Window offsets	from the training split; generator seeded with (data order, step), same for every optimizer
Steps	10,000
Tokens	328M (about 0.6 epochs)
Learning rate	$\eta\,\varphi(t)$ at step t, every parameter group
Warmup	linear, $\varphi(t) = (t+1)/200$ for $t < 200$
Decay	cosine to $\varphi = 0.1$ at step 10,000
Gradient clipping	none
Initialization seed	0, all runs
Data orders	0–4
Training precision	bfloat16 autocast
Prediction, Hessian precision	float32, no autocast (TF32 matrix products allowed)
Logged predictions	next-token distributions: 128 fixed validation sequences × 16 evenly spaced positions (2,048), as float16 log-probabilities
Loss for matching runs	64 fixed training sequences, all positions
Both logged at	every 250th step + 30 log-spaced steps in $[1, 10{,}000]$
Hessian	top eigenvalue, 20 Lanczos iterations, 32 fixed training sequences; AdamW also of $P^{-1/2}HP^{-1/2}$
Measured at	every 500th step + 30 log-spaced steps

Table 2: Per optimizer, TinyStories-10M runs.

	SGD	AdamW	Muon
η	0.03	$2 \cdot 10^{-3}$	0.01
η choice	next lower of a 5-rate sweep on d0; the best (0.1) blew up on d1–d3	best final validation loss of a 5-rate sweep on d0	as AdamW
Update	heavy ball, not Nesterov	bias-corrected moments	Nesterov momentum; 5 quintic Newton–Schulz steps in bfloat16
Momentum	$\mu = 0.9$	$\beta = (0.9, 0.95)$	$\beta = 0.95$
ϵ	–	10^{-8}	–
Update scale	–	–	$\sqrt{\max(1, n_{\text{out}}/n_{\text{in}})}$
Weight decay	none	0.1, decoupled; none on LayerNorm gains	none
Parameters	all	all	query, key, value, attention-output and both MLP matrices of every block
Aux. AdamW	–	–	embeddings, head, LayerNorm gains
Aux. η, β, ϵ	–	–	$2 \cdot 10^{-3}$, $(0.9, 0.95)$, 10^{-8}
Aux. weight decay	–	–	none

A.2 Runs of Figure 2

Table 3: Settings of Figure 2: rows that differ between settings; all else as in Appendix A.1. L layers, width d, vocabulary V; batch is sequences × tokens; "aux.": Muon's auxiliary AdamW.

	Shakespeare-char	TinyStories-10M (as Appendix A.1)	TinyStories-30M	Shakespeare, 8000 steps (memorization control)
Runs	3 optimizers × d0–d4 (15)	3 optimizers × d0–d4 (15)	3 optimizers × d0–d4 (15)	AdamW, Muon × d0–d4 (10)
Data	Shakespeare, nanoGPT's 90/10 split	TinyStories	TinyStories	as Shakespeare-char
V	65 (characters)	4096 (BPE)	4096 (BPE)	65 (characters)
L, heads, d	6, 6, 384	6, 6, 384	8, 8, 512	6, 6, 384
Parameters (without embeddings, head)	10.8M (10.6M)	13.9M (10.6M)	29.5M (25.2M)	10.8M (10.6M)
Batch	64 × 256	128 × 256	128 × 256	64 × 256
Steps	1,500	10,000	20,000	8,000
Tokens	24.6M (about 24 passes over the 1M training characters)	328M (about 0.6 epochs)	655M (about 1.2 epochs)	131M
Warmup steps w	100	200	400	100
Cosine decay to step	1,500	10,000	20,000	8,000 (stretched)
η_{SGD}	0.05	0.03	0.1	–
η_{AdamW}	$3 \cdot 10^{-4}$	$2 \cdot 10^{-3}$	$2 \cdot 10^{-3}$	$3 \cdot 10^{-4}$
η_{Muon} (aux.)	0.04 (10^{-3})	0.01 ($2 \cdot 10^{-3}$)	0.01 ($1.5 \cdot 10^{-3}$)	0.04 (10^{-3})
η chosen by	minimum validation loss over training (runs overfit)	best final validation loss	best final validation loss	as Shakespeare-char
SGD's best η, diverged*	0.2	0.1 (d1–d3)	0.3	–
AdamW β	$(0.9, 0.99)$, as nanoGPT	$(0.9, 0.95)$	$(0.9, 0.95)$	$(0.9, 0.99)$
Aux. β	$(0.9, 0.95)$	$(0.9, 0.95)$	$(0.9, 0.95)$	$(0.9, 0.95)$
Predictions	all 256 positions of 64 validation sequences (16,384)	16 positions of 128 sequences (2,048)	16 positions of 128 sequences (2,048)	as Shakespeare-char
Predictions at	every 50th + 40 log-spaced	every 250th + 30 log-spaced	every 500th + 30 log-spaced	every 100th + 40 log-spaced
Hessian at	every 50th + 30 log-spaced	every 500th + 30 log-spaced	every 1000th + 30 log-spaced	none

*On other data orders; SGD then uses a lower rate.

A.3 Runs of Figure 3

Table 4: Swap forks. A: parent's optimizer; B: optimizer after the swap; s: fork step.

Parents	the three d0 runs of Appendix A.1
Shadow states	the other two optimizers, stepped on the same mini-batch gradients with learning rate 0: momenta, second moments and step counts evolve as if training; parameters follow A alone
Fork steps s	50, 100, 200, 400, 700, 1000, 1500, 2500, 4000
Saved at s	parameters; states of all three optimizers
Swap $A \to B$ loads	parameters at s; B's shadow state
Hyperparameters	B's (Table 2)
Steps	s to 10,000
Schedule	the parent's; learning rate $\eta_B \, \varphi(s)$ at s
Batches	the parent's (data order 0)
Model, probes, logging grids	the parent's
Swap forks	6 ordered pairs × 9 steps = 54
Same-batch control	the parent itself (A continued with the same batches)
Data-order control	A continued from s with its own state, data order 1: 27 forks
Total forks	81